Sauvegardes

Candex sauvegarde sa base **PostgreSQL de production**. Ce handbook couvre la stratégie, les scripts, comment restaurer, et comment c'est automatisé. Pour où vit la base, voir Base de données & Modèle ; pour les environnements, Déploiement.

> **Note**
>
> Le dépôt `candex-backups` est l'ancien outil de l'ère Firestore et ne sauvegarde plus de vraies données. Les sauvegardes actuelles sont des dumps PostgreSQL pilotés depuis `candex-api/scripts/backup/`.

Stratégie

Seule la **prod** (`candex_prod`) est sauvegardée — la dev sert de terrain d'essai. L'approche : un dump complet quotidien avec rétention grandfather-father-son (GFS), plus un instantané permanent par release :

- **Quotidien** — dumps complets — `KEEP_DAILY = 7`
- **Hebdomadaire** — le dump du dimanche est promu — `KEEP_WEEKLY = 5`
- **Mensuel** — le dump du 1er du mois est promu — `KEEP_MONTHLY = 12`
- **Par version** — un instantané labellisé (`--label x.y.z`), **gardé à vie, jamais élagué**

Sauvegardes complètes — `daily_backup.sh`

Le script principal exécute un `pg_dump -Fc` (format custom : compressé et restaurable sélectivement) de toute la base, à travers le conteneur Postgres en cours :

```
./scripts/backup/daily_backup.sh prod                 # dump complet quotidien
./scripts/backup/daily_backup.sh prod --label 1.1.9   # + un instantané de version conservé
```

Chaque exécution écrit dans `backups/<env>/full/`, valide le dump (non vide et passant `pg_restore --list`), puis le promeut dans les paliers `daily`, `weekly`, `monthly` et `version` selon le cas :

```
backups/prod/full/
├── daily/     candex_prod_<timestamp>.dump
├── weekly/    (dimanches)
├── monthly/   (1er du mois)
└── version/   candex_prod_v1.1.9.dump   (gardé à vie)
```

Le dossier `backups/` est git-ignoré — les dumps ne vont jamais dans git.

Exports sélectifs — `dump_*.sh`

En plus des dumps complets, des exports **data-only** par domaine couvrent les données runtime difficiles à reconstruire — `dump_catalog_data.sh`, `dump_dex_data.sh`, `dump_user_data.sh` (et `dump_all_runtime_data.sh`). Les données de référence statiques sont volontairement exclues, car Flyway les seede (voir Base de données & Modèle).

Restauration & test

Une sauvegarde ne vaut que par une restauration testée. `restore.sh` restaure un dump dans une base **jetable** pour la vérifier, puis la supprimer :

```
./scripts/backup/restore.sh <fichier.dump> candex_restore_test
# vérifier, puis : dropdb candex_restore_test
```

`pg_restore` doit être de la même version que le serveur ou plus récent.

Automatisation (CI)

Les sauvegardes tournent sur le runner self-hosted présent sur le serveur de production, via GitHub Actions :

Workflow	Quand	Quoi
`backup-prod.yml`	Quotidien à **03:00 UTC** (et manuel)	`daily_backup.sh prod`
`deploy-prod.yml`	Avant chaque déploiement prod	`daily_backup.sh prod --label <version>` — un point de reprise pré-déploiement

Aucun cron OS nécessaire — la planification vit dans le workflow.

Stockage & hors-site

Les dumps sont gardés **sur le serveur** pour l'instant ; les sortir de l'hôte (un second disque ou `rsync` vers une autre machine) est une amélioration prévue. Une variante autonome sous `ops/backup/` (un timer `systemd` plus copie hors-site `rsync` et alerting en cas d'échec) est disponible pour ce setup côté serveur.

Conventions

- **Aucun secret dans git** — le mot de passe Postgres vit dans `~/.pgpass` (`chmod 600`) ; la config locale (`backup.conf`) et les dumps `backups/` ne sont jamais committés.
- **Complet + sélectif** — les dumps complets `-Fc` sont le chemin de restauration ; les exports par domaine sont des instantanés de confort.
- **Tester les restaurations** — un dump jamais restauré n'est pas une sauvegarde.

Où aller ensuite

- Où vivent les données → Base de données & Modèle
- Environnements & CI → Déploiement